

Chatbot Tradicional · Flujos sin IA

FastChat DJ · Plataforma WhatsApp CRM con IA

Generado automáticamente desde la fuente `.md`
Versión actualizada · Abril 2026

Chatbot tradicional (flujo tipo n8n)

Motor de flujos conversacionales con menús, captura de datos, llamadas HTTP a APIs externas, condicionales y handoff humano. Convive con el agente IA existente sin interferir.

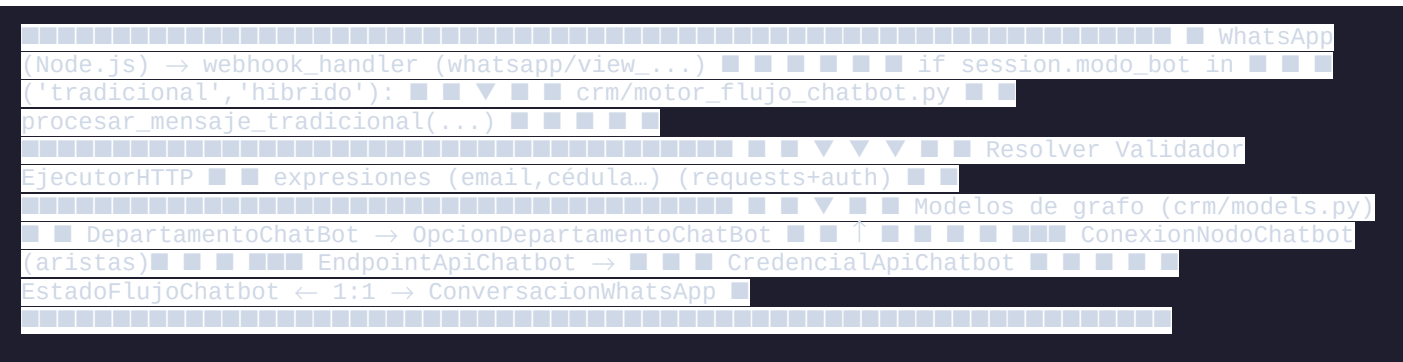
1. Para ejecutar / probar rápido

```
# 1) Aplicar migraciones (crea modelos nuevos) python manage.py migrate # 2) Sembrar un flujo de ejemplo (Centro de Atención Estudiantil) python manage.py seed_centro_estudiantil # Opciones: python manage.py seed_centro_estudiantil --reset # borra y recrea python manage.py seed_centro_estudiantil --sesion <id> # asocia a una SesionWhatsApp # y setea modo_bot='tradicional' # 3) Activar el motor en una sesión manualmente (si no usaste --sesion) # Admin de Django → SesionWhatsApp → elige la sesión → # modo_bot = 'tradicional' (o 'hibrido') # departamento_default = Centro de Atención Estudiantil # departamentos (M2M) agregar el mismo depto
```

Script que llena las plantillas base: `crm/management/commands/seed_centro_estudiantil.py`

Se invoca con: `python manage.py seed_centro_estudiantil`

2. Arquitectura en capas



3. Modelos (qué guarda cada uno)

Modelo	Rol
--------	-----

DepartamentoChatBot	Workflow completo. Campos: <code>nombre</code> , <code>mensaje_saludo</code> , <code>palabras_clave</code> , <code>es_default</code> , <code>activo_tradicional</code> .
OpcionDepartamentoChatBot	Nodo del grafo. Campos clave: <code>tipo_nodo</code> , <code>config</code> (JSON), <code>endpoint</code> , <code>variable_destino</code> , <code>validacion_tipo</code> , <code>reintentos_max</code> , <code>es_inicio</code> . Mantiene <code>opcion_padre</code> como fallback legacy (árbol).
ConexionNodoChatbot	Arista origen→destino con <code>etiqueta</code> (vacía=default, <code>true/false</code> , <code>ok/error</code> , <code>opcion_X</code> , <code>timeout...</code>).
CredencialApiChatbot	Credencial reusable (<code>none/bearer/basic/apikey_header/apikey_query/custom_header</code>). Secretos en JSON.
EndpointApiChatbot	Endpoint HTTP reusable: <code>base_url</code> , <code>credencial</code> , <code>headers_default</code> , <code>timeout_seg</code> .
EstadoFlujoChatbot	1:1 con <code>ConversacionWhatsApp</code> . Guarda <code>nodo_actual</code> , <code>variables</code> (JSON), <code>intentos</code> , <code>finalizado</code> .

Switch global en `whatsapp.SesionWhatsApp`: - `modo_bot`: `ia` (default) / `tradicional` / `hibrido` / `ninguno` - `departamento_default`: depto al que va si no hay match por palabras clave.

4. Tipos de nodo

Tipo	Qué hace	Config principal	Etiquetas de salida
<code>menu</code>	Lista opciones, espera elección por número/texto	<code>{mensaje, opciones: [{etiqueta, valor, opcion, timeout}]}</code>	la <code>salida</code> de cada opción
<code>pregunta</code>	Pide dato, valida, guarda en <code>variable_destino</code>	<code>{pregunta} + validacion_tipo</code> del nodo	<code>''</code> ok, <code>timeout</code> tras N fallos
<code>respuesta</code>	Envía texto y avanza	<code>{mensaje}</code>	<code>''</code>
<code>http</code>	Llama API externa	<code>{metodo, path, query, body, extraer[], plantilla_respuesta} + endpoint</code> FK	<code>ok</code> , <code>error</code>

condicional	If/Else	{operador: and/or, condiciones:[{izq,op,der}]}	true, false
switch	Branch por valor	{valor, casos:[{match,salida}]}	salida del caso o default
set_variable	Asigna variables	{asignaciones:[{variable,'<presion}]}	
handoff	Transfiere a humano	{mensaje}	termina turno
esperar	Delay	{segundos}	' '
fin	Cierra flujo	{mensaje}	marca finalizado=True
inicio	Arranque explícito	—	' '

Operadores de condicional

==, !=, >, <, >=, <=, contiene, no_contiene, vacio, no_vacio.

Validaciones de pregunta

email, numero, telefono, fecha (YYYY-MM-DD), cedula (EC, algoritmo verificador), ruc, regex (con validacion_expresion).

5. Expresiones {{...}} (resolver)

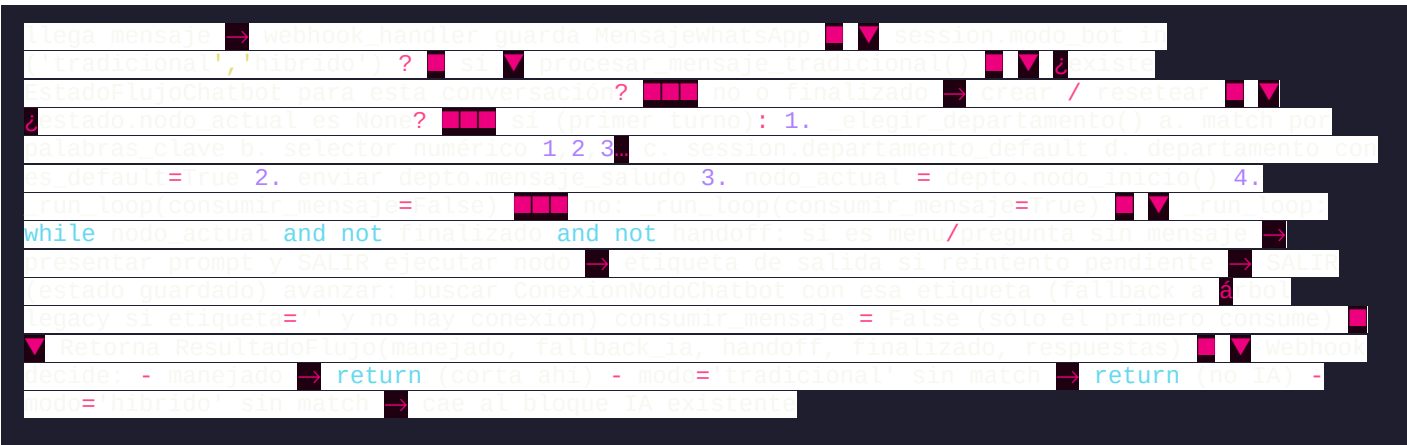
En cualquier string de config puedes usar placeholders. Contexto disponible:

Expresión	Valor
{{variables.X}}	Variable capturada o extraída (cédula, respuesta HTTP...)
{{contacto.numero}}	Número del contacto
{{contacto.nombre}}	Nombre del contacto
{{conversacion.id}}	ID de la conversación

<code>{{sesion.numero}} / {{sesion.nombre}} / {{sesion.session_id}}</code>	Datos de la sesión
<code>{{mensaje.texto}}</code>	Último mensaje del usuario

Soporta paths anidados: `{{_last_http.body.data[0].nombre}}`. Si la cadena completa es una expresión (`"{{variables.n}}"`), se preserva el tipo original (útil para JSON bodies con números/booleans).

6. Ciclo de vida de un mensaje



Dato clave: un solo mensaje puede ejecutar una cadena larga de nodos (ej: `pregunta` → `http` → `set_variable` → `condicional` → `respuesta` → `fin`), porque todos los que no requieren input corren en cascada hasta topar con `menu/pregunta` o con `fin/handoff`.

Tope de seguridad: `MAX_NODOS_POR_TURN0 = 25`.

7. Matriz `modo_bot`

Modo	Motor tradicional	Agente IA	Caso de uso
<code>ia</code> (default)	■	■	Comportamiento previo, sin cambios
<code>tradicional</code>	■	■	Flujo estricto con APIs
<code>hibrido</code>	■	■ si motor no matcheó	FAQ por flujo, el resto a IA

ninguno	■	■	Sólo humanos
---------	---	---	--------------

El bloque nuevo en `whatsapp/view_webhook_handler.py` **corta** antes de llegar al pipeline IA cuando el motor manejó la conversación. Genera trazas `etapa='motor_flujo'`.

8. Recetas de **config** por tipo de nodo

menu

```
"mensaje": "¿Qué deseas?", "opciones": [{"etiqueta": "Consultar saldo", "valor": "saldo",
"salida": "saldo"}, {"etiqueta": "Hablar con asesor", "valor": "asesor", "salida": "asesor"}]
```

Conexiones: `ConexionNodoChatbot(origen=menu, destino=nodo_saldo, etiqueta='saldo')`, etc.

pregunta

```
"pregunta": "Dime tu cédula (10 dígitos):"
```

Campos del nodo: `variable_destino='cedula'`, `validacion_tipo='cedula'`, `reintentos_max=3`, `mensaje_error='Cédula inválida'`.

http

```
"metodo": "POST", "path": "/clientes/{{variables.cedula}}/saldo", "query": "periodo=2026-1", "body": {"cedula": "{{variables.cedula}}", "canal": "whatsapp"}, "extraer": {"variable": "saldo", "jsonpath": "data.saldo"}, {"variable": "cliente", "jsonpath": "data.nombre"}, "plantilla_respuesta": "Hola {{variables.cliente}}, tu saldo es ${{variables.saldo}}"
```

Campo del nodo: `endpoint` FK al `EndpointApiChatbot`. Salidas: **ok** (2xx) / **error** (no-2xx, timeout, conexión).

condicional

```
"operador": "and", "condiciones": [{"izq": "{{variables.saldo}}", "op": ">", "der": "0"}, {"izq": "{{variables.tipo}}", "op": "contiene", "der": "VIP"}]
```

switch

```
"valor": "{{variables.tipo_cliente}}" "casos": [ {"match": "vip", "salida": "vip"},  
"match": "regular", "salida": "regular"} ] }
```

set_variable

```
"asignaciones": [ {"variable": "saludo_fmt", "expresion": "Hola {{contacto.nombre}}",  
"variable": "timestamp", "expresion": "2026-04-15"} ] }
```

handoff

```
"mensaje": "Te conecto con un asesor..."
```

fin

```
"mensaje": "Gracias por tu consulta."
```

9. Credenciales (autenticación de APIs)

`CredencialApiChatbot.secretos` es un JSON cuyo shape depende de `tipo`:

Tipo	Shape de secretos
<code>none</code>	<code>{}</code>
<code>bearer</code>	<code>{"token": "eyJ..."}</code>
<code>basic</code>	<code>{"usuario": "x", "password": "y"}</code>
<code>apikey_header</code>	<code>{"nombre_header": "X-API-Key", "valor": "abc123"}</code>
<code>apikey_query</code>	<code>{"nombre_param": "api_key", "valor": "abc123"}</code>
<code>custom_header</code>	<code>{"headers": {"X-Tenant": "a", "X-Env": "prod"}}</code>

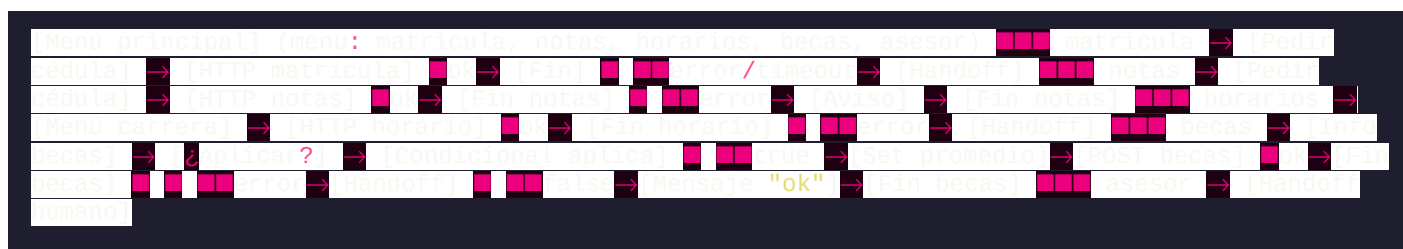
Se guardan en texto plano (igual que `ApiKeyIA.descripcion` del módulo IA). Si se necesita cifrado más adelante, migrar a un campo cifrado y adaptar `_aplicar_credencial()` en el motor.

10. Ejemplo: el seed del Centro Estudiantil

`crm/management/commands/seed_centro_estudiantil.py` crea:

- 1 departamento `Centro de Atención Estudiantil` (`es_default=True`).
- 1 credencial demo + 2 endpoints demo (`jsonplaceholder`, `httpbin`).
- 18 nodos, 23 conexiones.

Grafo:



Ejemplos demostrados en el seed: - Validación real de cédula ecuatoriana. - GET con query templates (`{{variables.cedula}}`). - POST con body templates (`{{contacto.numero}}`, `{{conversacion.id}}`). - Extracción JSONPath (`args.cedula`, `address.city`). - Condicional `or` con `contiene`. - `set_variable` inyectando dato antes de un POST. - Rama `timeout` cuando se agotan reintentos. - Rama `error` cuando el HTTP falla.

Flujo de prueba end-to-end

1. WhatsApp → `hola` → menú principal.
2. → `1` → pide cédula.
3. → `1710034065` → valida, hace GET a `httpbin`, extrae echo, responde `"Matrícula ACTIVA..."`, cierra flujo.

11. Archivos principales

Archivo	Rol
<code>crm/models.py</code>	Modelos del grafo (extiende <code>OpcionDepartamentoChatBot</code> , agrega <code>Credencial/Endpoint/Conexion/Estado</code>)

<code>crm/motor_flujo_chatbot.py</code>	Motor de ejecución (único entry: <code>procesar_mensaje_tradicional</code>)
<code>crm/admin.py</code>	Admin de Django para editar el grafo manualmente
<code>crm/management/commands/seed_centro_estudiantil.py</code>	Seed base — comando a ejecutar para plantillas
<code>whatsapp/view_webhook_handler.py</code>	Enganche del motor (bloque nuevo antes del pipeline IA)
<code>whatsapp/models.py</code>	<code>SesionWhatsApp.modelo_bot</code> + <code>departamento_default</code>

12. Edición manual de flujos

Mientras no exista editor visual:

- **Admin de Django** (`/admin/crm/`):
- **Departamentos ChatBot** → crear + `es_default`, palabras clave.
- **Nodos de Flujo ChatBot** → crear cada nodo, `tipo_nodo`, `config` (JSON), marcar `es_inicio` en el primero.
- **Conexiones entre nodos** → aristas con `etiqueta`.
- **Endpoints API ChatBot / Credenciales API ChatBot** → catálogo de APIs externas.

Sesiones WhatsApp → `modo_bot`, `departamento_default`, M2M `departamentos`.

Seed programático: replicar el patrón de `seed_centro_estudiantil.py` (`_nodo()` y `_conectar()`), es la forma más rápida para flujos medianos/grandes.

13. Extensiones futuras (no hechas)

- Cifrado de `CredencialApiChatbot.secretos`.
- Editor visual del grafo (drag-and-drop).
- Nodo `ia` para delegar a agente IA a mitad del flujo y volver.
- Retries/backoff automáticos en nodo `http`.
- Plantillas de flujos exportables (JSON).